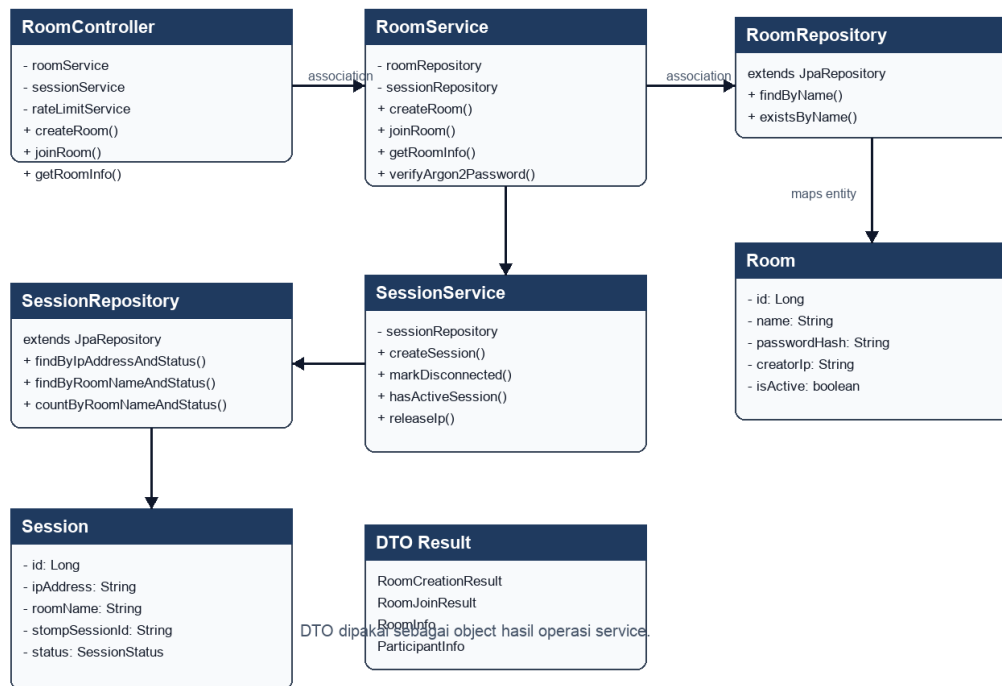


Class Diagram - Manhattan

Class Diagram - Backend Manhattan



Hubungan Class

Association: controller memakai service, service memakai repository, repository mengakses entity.
Inheritance: repository extends JpaRepository. Composition/agregasi: service menyimpan dependency repository melalui constructor injection.

Screenshot Kode

Class Evidence - RoomService Dependencies

```
18 import java.util.List;
19 import java.util.Optional;
20 import java.util.regex.Pattern;
21
22 @Service
23 public class RoomService {
24
25     private static final int MAX_PARTICIPANTS = 50;
26     private static final Pattern ROOM_NAME_PATTERN = Pattern.compile("[a-zA-Z0-9]{3,15}");
27
28     private final RoomRepository roomRepository;
29     private final SessionRepository sessionRepository;
30
31     public RoomService(RoomRepository roomRepository, SessionRepository sessionRepository) {
32         this.roomRepository = roomRepository;
33         this.sessionRepository = sessionRepository;
34     }
35
36     /**
37      * Validates a room name against the naming rules:
38      * - Alphanumeric only (case-sensitive)
39      * - No spaces
40      * - Between 3 and 15 characters
41      */
42     public boolean validateRoomName(String roomName) {
43         if (roomName == null) {
44             return false;
45         }
46         return ROOM_NAME_PATTERN.matcher(roomName).matches();
47     }
48
49     /**
50      * Creates a new room with the given name and optional password hash.
51      * The password hash is received already hashed (Argon2id PHC format) from the client.
52      */
53     public RoomCreationResult createRoom(String roomName, String passwordHash, String creatorIp) {
54         if (!validateRoomName(roomName)) {
55             return RoomCreationResult.failure("Invalid room name. Must be alphanumeric, 3-15 characters, no spaces.");
56         }
57
58         if (roomRepository.existsByName(roomName)) {
59             return RoomCreationResult.failure("Room name is already taken.");
60         }
61
62         Room room = new Room();
63         room.setName(roomName);
64         room.setPasswordHash(passwordHash);
65         room.setCreatorIp(creatorIp);
66         room.setCreatedAt(LocalDate.now());
67         room.setActive(true);
68
69         roomRepository.save(room);
70
71         return RoomCreationResult.success(roomName);
72     }
73
74     /**
75      * Joins an existing room. For password-protected rooms, the client sends the plaintext
76      * password and the server verifies it against the stored Argon2id hash using Bouncy Castle.
77      *
78      * @param roomName the room to join
79      * @param password the plaintext password (null for rooms without password)
80      * @param clientIp the IP address of the joining client
81      */
82     public RoomJoinResult joinRoom(String roomName, String password, String clientIp) {
83         Optional<Room> roomOpt = roomRepository.findByName(roomName);
84
85         if (roomOpt.isEmpty()) {
86             return RoomJoinResult.failure("Room not found.");
87         }
88     }
```

Repository Evidence - SessionRepository

```
1 package com.manhattan.repository;
2
3 import com.manhattan.entity.Session;
4 import com.manhattan.entity.SessionStatus;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.stereotype.Repository;
7
8 import java.util.List;
9 import java.util.Optional;
10
11 @Repository
12 public interface SessionRepository extends JpaRepository<Session, Long> {
13
14     Optional<Session> findByIpAddressAndStatus(String ipAddress, SessionStatus status);
15
16     List<Session> findByRoomNameAndStatus(String roomName, SessionStatus status);
17
18     long countByRoomNameAndStatus(String roomName, SessionStatus status);
19 }
```

DTO Evidence - RoomJoinResult

```
1 package com.manhattan.dto;
2
3 public class RoomJoinResult {
4
5     private final boolean success;
6     private final String message;
7     private final int participantCount;
8
9     public RoomJoinResult(boolean success, String message, int participantCount) {
10         this.success = success;
11         this.message = message;
12         this.participantCount = participantCount;
13     }
14
15     public static RoomJoinResult success(int participantCount) {
16         return new RoomJoinResult(true, "Joined room successfully", participantCount);
17     }
18
19     public static RoomJoinResult failure(String message) {
20         return new RoomJoinResult(false, message, 0);
21     }
22
23     public boolean isSuccess() {
24         return success;
25     }
26
27     public String getMessage() {
28         return message;
29     }
30 }
```